```
import pandas as pd
import numpy as np
```

```
df = pd.read_csv('/content/Sports_Analytics_100_Records.csv')
df.head()
```

	Player_ID	Player_Name	Team_Name	Gender	Age	Fitness_Level	Matches_Played	Runs_Scored	Opponent_Strength	Match_Conditior
0	1	Aarav Sharma	Thunder Strikers	Male	22	8	25	780	6	7
1	2	Vivaan Patel	Royal Blazers	Male	27	7	30	920	8	6
2	3	Aditya Rao	Phoenix Warriors	Male	24	9	28	1100	5	8
3	4	Arjun Mehta	Titan Chargers	Male	29	6	22	640	7	5
4	5	Sai Reddy	Coastal Kings	Male	21	8	18	540	4	9

Next steps: Generate code with df New interactive sheet

```
df = df.drop(['Player_ID', 'Player_Name'], axis=1)
```

```
from sklearn.preprocessing import LabelEncoder

le = LabelEncoder()

categorical_cols = ['Team_Name', 'Gender', 'Match_Condition']

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])
```

```
X = df.drop('Future_Performance', axis=1)
y = df['Future_Performance']
```

```
from sklearn.model_selection import train_test_split

X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
from sklearn.preprocessing import StandardScaler

scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
from sklearn.linear_model import LinearRegression
from sklearn.ensemble import RandomForestRegressor

lr = LinearRegression()
lr.fit(X_train, y_train)

rf = RandomForestRegressor(n_estimators=100, random_state=42)
rf.fit(X_train, y_train)
```

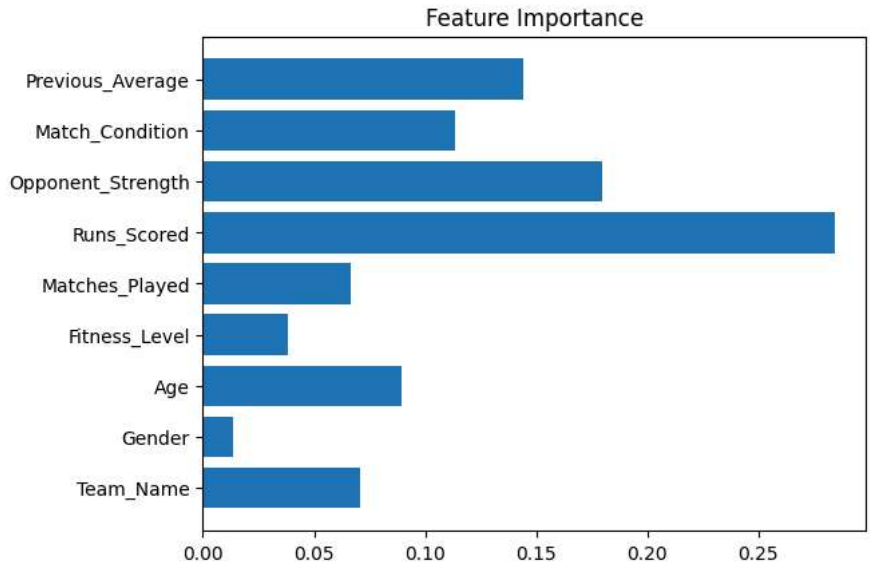
```
RandomForestRegressor
RandomForestRegressor(random_state=42)
```

```
lr_pred = lr.predict(X_test)
rf_pred = rf.predict(X_test)
```

```
from sklearn.metrics import mean_squared_error, r2_score

print("Linear Regression:")
print("MSE:", mean_squared_error(y_test, lr_pred))
print("R2:", r2_score(y_test, lr_pred))

print("\nRandom Forest:")
print("MSE:", mean_squared_error(y_test, rf_pred))
print("R2:", r2_score(y_test, rf_pred))
```

```
Linear Regression:
MSE: 19.490288896223387
R2: 0.015020144221180698

Random Forest:
MSE: 13.026240000000007
R2: 0.34169349336702426
```

```
import matplotlib.pyplot as plt

feature_importance = rf.feature_importances_
features = X.columns

plt.barh(features, feature_importance)
plt.title("Feature Importance")
plt.show()
```



Gradient Boosting Regressor

```
from sklearn.ensemble import GradientBoostingRegressor

gbr = GradientBoostingRegressor(n_estimators=100, random_state=42)
gbr.fit(X_train, y_train)

gbr_pred = gbr.predict(X_test)

print("\nGradient Boosting:")
print("MSE:", mean_squared_error(y_test, gbr_pred))
print("R2:", r2_score(y_test, gbr_pred))
```

```
Gradient Boosting:
MSE: 11.836993236032676
R2: 0.4017944037380834
```

```
models = {
    "Linear Regression": lr_pred,
    "Random Forest": rf_pred,
    "Gradient Boosting": gbr_pred,

}
```

```
print("\nModel Comparison:")
for name, pred in models.items():
    print(f"{name} -> R2 Score: {r2_score(y_test, pred):.4f}")
```

```
Model Comparison:
Linear Regression -> R2 Score: 0.0150
Random Forest -> R2 Score: 0.3417
Gradient Boosting -> R2 Score: 0.4018
```

```
from google.colab import files
uploaded = files.upload()
```

Choose files Sports_Anal…Records.csv

Sports_Analytics_100_Records.csv(text/csv) - 6038 bytes, last modified: 01/04/2026 - 100% done

```
Saving Sports_Analytics_100_Records.csv to Sports_Analytics_100_Records (2).csv
```

```
import pandas as pd
import numpy as np

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import LabelEncoder, StandardScaler

import tensorflow as tf
from tensorflow import keras
from tensorflow.keras import layers
```

```
df = pd.read_csv("Sports_Analytics_100_Records.csv")

df.head()
```

	Player_ID	Player_Name	Team_Name	Gender	Age	Fitness_Level	Matches_Played	Runs_Scored	Opponent_Strength	Match_Conditior
0	1	Aarav Sharma	Thunder Strikers	Male	22	8	25	780	6	7
1	2	Vivaan Patel	Royal Blazers	Male	27	7	30	920	8	6
2	3	Aditya Rao	Phoenix Warriors	Male	24	9	28	1100	5	8
3	4	Arjun Mehta	Titan Chargers	Male	29	6	22	640	7	5
4	5	Sai Reddy	Coastal Kings	Male	21	8	18	540	4	9

Next steps: Generate code with df New interactive sheet

```
df.info()
```

```
<class 'pandas.core.frame.DataFrame'>
RangeIndex: 100 entries, 0 to 99
Data columns (total 12 columns):
 #   Column              Non-Null Count  Dtype
---  ------              --------------  -----
 0   Player_ID           100 non-null    int64
 1   Player_Name         100 non-null    object
 2   Team_Name           100 non-null    object
 3   Gender              100 non-null    object
 4   Age                 100 non-null    int64
 5   Fitness_Level       100 non-null    int64
 6   Matches_Played      100 non-null    int64
 7   Runs_Scored         100 non-null    int64
 8   Opponent_Strength   100 non-null    int64
 9   Match_Condition     100 non-null    int64
 10  Previous_Average    100 non-null    float64
 11  Future_Performance  100 non-null    int64
dtypes: float64(1), int64(8), object(3)
memory usage: 9.5+ KB
```

```
df = df.drop(['Player_ID', 'Player_Name'], axis=1)
```

```
le = LabelEncoder()

categorical_cols = ['Team_Name', 'Gender', 'Match_Condition']

for col in categorical_cols:
    df[col] = le.fit_transform(df[col])

df.head()
```

	Team_Name	Gender	Age	Fitness_Level	Matches_Played	Runs_Scored	Opponent_Strength	Match_Condition	Previous_Average	Futu
0	6	1	22	8	25	780	6	3	31.2	
1	5	1	27	7	30	920	8	2	30.6	
2	4	1	24	9	28	1100	5	4	39.2	
3	7	1	29	6	22	640	7	1	29.1	
4	0	1	21	8	18	540	4	5	30.0	

Next steps: Generate code with df New interactive sheet

```
X = df.drop("Future_Performance", axis=1)
y = df["Future_Performance"]
```

```
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)
```

```
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

```
model = keras.Sequential([

    layers.Dense(64, activation='relu', input_shape=(X_train.shape[1],)),

    layers.Dense(32, activation='relu'),

    layers.Dense(16, activation='relu'),

    layers.Dense(1)
])
```

```
/usr/local/lib/python3.12/dist-packages/keras/src/layers/core/dense.py:106: UserWarning: Do not pass an `input_shape`/`input_dim
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

```
model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

```
history = model.fit(
    X_train,
    y_train,
    epochs=100,
    batch_size=8,
    validation_split=0.2
)
```

Epoch 77/100

```
Epoch 77/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 16ms/step - loss: 8.9386 - mae: 2.3145 - val_loss: 63.4289 - val_mae: 6.5468
Epoch 78/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 8.6936 - mae: 2.2787 - val_loss: 63.7479 - val_mae: 6.5724
Epoch 79/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 8.6821 - mae: 2.3039 - val_loss: 65.1685 - val_mae: 6.6527
Epoch 80/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 8.5302 - mae: 2.2723 - val_loss: 65.1467 - val_mae: 6.6359
Epoch 81/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 8.4290 - mae: 2.2371 - val_loss: 62.7848 - val_mae: 6.5118
Epoch 82/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 8.2507 - mae: 2.2380 - val_loss: 62.8602 - val_mae: 6.5253
Epoch 83/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 8.0997 - mae: 2.2091 - val_loss: 63.2894 - val_mae: 6.5604
Epoch 84/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 8.0170 - mae: 2.1835 - val_loss: 62.8366 - val_mae: 6.5344
Epoch 85/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 7.7893 - mae: 2.1565 - val_loss: 62.1530 - val_mae: 6.5015
Epoch 86/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 7.8010 - mae: 2.1652 - val_loss: 62.5331 - val_mae: 6.5118
Epoch 87/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 7.4734 - mae: 2.1109 - val_loss: 61.9091 - val_mae: 6.4805
Epoch 88/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 7.4941 - mae: 2.0844 - val_loss: 62.0838 - val_mae: 6.4899
Epoch 89/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 7.5401 - mae: 2.0950 - val_loss: 61.2988 - val_mae: 6.4437
Epoch 90/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step - loss: 7.1474 - mae: 2.0604 - val_loss: 60.5591 - val_mae: 6.3955
Epoch 91/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 7.1501 - mae: 2.0521 - val_loss: 60.8558 - val_mae: 6.4190
Epoch 92/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 6.8797 - mae: 2.0093 - val_loss: 61.0790 - val_mae: 6.4477
Epoch 93/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 7.0006 - mae: 2.0596 - val_loss: 61.2656 - val_mae: 6.4462
Epoch 94/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 6.7246 - mae: 2.0039 - val_loss: 61.1842 - val_mae: 6.4607
Epoch 95/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 6.5869 - mae: 1.9669 - val_loss: 61.2684 - val_mae: 6.4552
Epoch 96/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 6.4268 - mae: 1.9376 - val_loss: 60.9342 - val_mae: 6.4348
Epoch 97/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 6.3389 - mae: 1.9338 - val_loss: 60.2591 - val_mae: 6.3923
Epoch 98/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 12ms/step - loss: 6.4434 - mae: 1.9471 - val_loss: 60.5686 - val_mae: 6.4278
Epoch 99/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 13ms/step - loss: 6.0738 - mae: 1.8736 - val_loss: 59.9692 - val_mae: 6.3960
Epoch 100/100
8/8 ━━━━━━━━━━━━━━━━━━━━ 0s 14ms/step - loss: 6.1643 - mae: 1.8928 - val_loss: 59.0444 - val_mae: 6.3364
```

```
loss, mae = model.evaluate(X_test, y_test)

print("Test MAE:", mae)
```

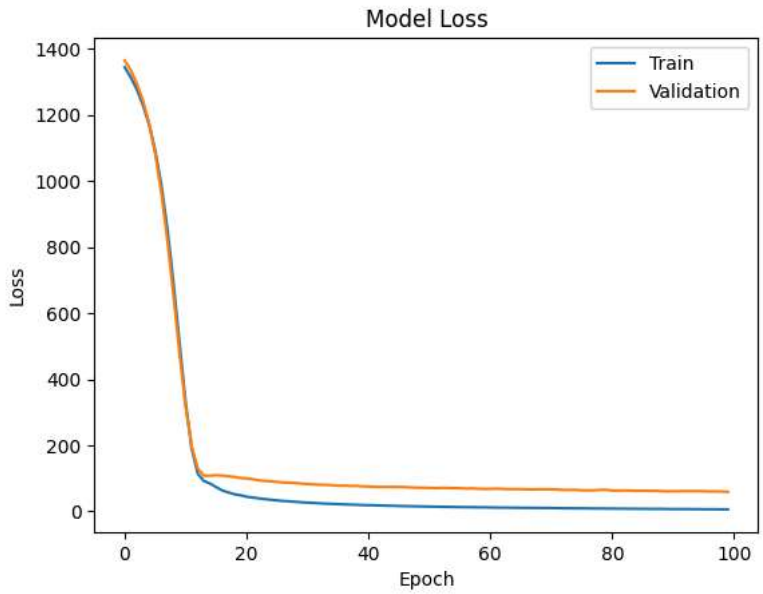
```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 187ms/step - loss: 134.1136 - mae: 8.4539
Test MAE: 8.453912734985352
```

```
predictions = model.predict(X_test)

print("Predicted Values:")
print(predictions[:10])

print("Actual Values:")
print(y_test[:10].values)
```

```
1/1 ━━━━━━━━━━━━━━━━━━━━ 0s 88ms/step
Predicted Values:
[[34.162292]
 [31.270561]
 [35.043312]
 [28.853306]
 [37.288937]
 [38.920982]
 [38.71413 ]
 [41.58235 ]
 [41.290283]
 [44.231606]]
Actual Values:
[38 38 40 30 44 39 37 35 40 34]
```

```
import matplotlib.pyplot as plt

plt.plot(history.history['loss'])
```

```
plt.plot(history.history['val_loss'])
plt.title("Model Loss")
plt.ylabel("Loss")
plt.xlabel("Epoch")
plt.legend(['Train', 'Validation'])
plt.show()
```




```
model.save("sports_performance_model.h5")
```

```
WARNING:absl:You are saving your model as an HDF5 file via `model.save()` or `keras.saving.save_model(model)`. This file format
```

```
from sklearn.linear_model import LinearRegression
from sklearn.tree import DecisionTreeRegressor
from sklearn.ensemble import RandomForestRegressor
from sklearn.svm import SVR

from sklearn.metrics import mean_absolute_error, mean_squared_error, r2_score

import pandas as pd

models = {
    "Linear Regression": LinearRegression(),
    "Decision Tree": DecisionTreeRegressor(),
    "Random Forest": RandomForestRegressor(n_estimators=100),
    "Support Vector Regressor": SVR()
}

results = []

for name, model in models.items():

    model.fit(X_train, y_train)

    predictions = model.predict(X_test)

    mae = mean_absolute_error(y_test, predictions)
    mse = mean_squared_error(y_test, predictions)
    r2 = r2_score(y_test, predictions)

    results.append([name, mae, mse, r2])

comparison_df = pd.DataFrame(results, columns=["Model", "MAE", "MSE", "R2 Score"])

print(comparison_df)
```

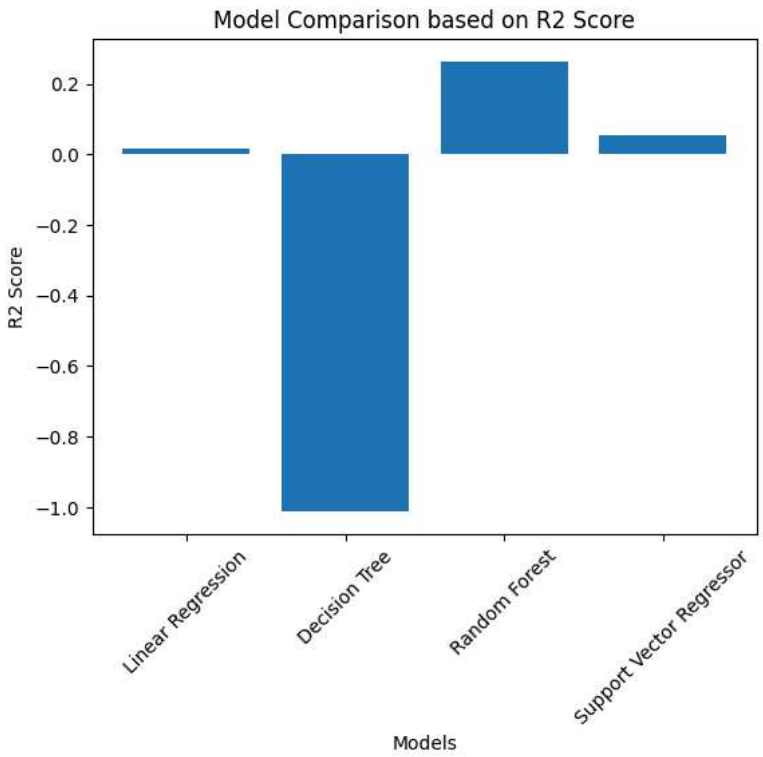
```
                      Model       MAE        MSE  R2 Score
0         Linear Regression  3.893071  19.490289  0.015020
1             Decision Tree  5.100000  39.800000 -1.011371
2             Random Forest  3.241000  14.585930  0.262872
3  Support Vector Regressor  3.869722  18.725803  0.053655
```

```
comparison_df
```

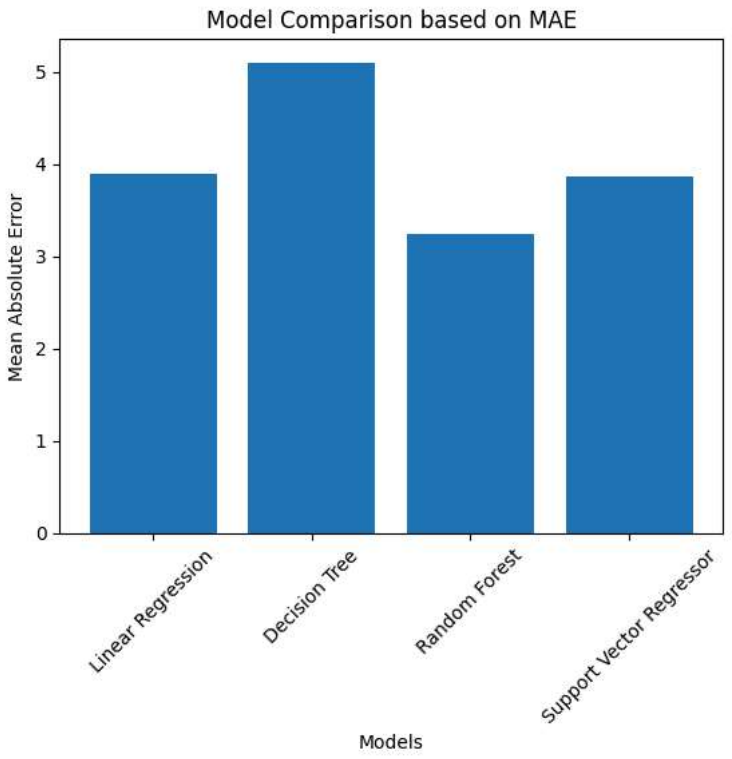
	Model	MAE	MSE	R2 Score
0	Linear Regression	3.893071	19.490289	0.015020
1	Decision Tree	5.100000	39.800000	-1.011371
2	Random Forest	3.241000	14.585930	0.262872
3	Support Vector Regressor	3.869722	18.725803	0.053655

Next steps: Generate code with comparison_df New interactive sheet

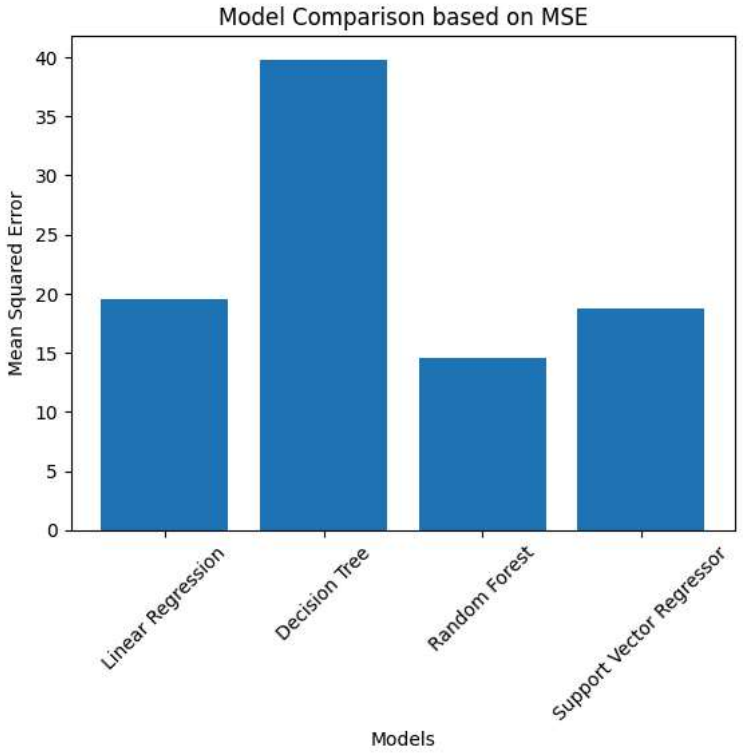
```
import matplotlib.pyplot as plt

plt.figure()

plt.bar(comparison_df["Model"], comparison_df["R2 Score"])

plt.xlabel("Models")
plt.ylabel("R2 Score")
plt.title("Model Comparison based on R2 Score")

plt.xticks(rotation=45)

plt.show()
```

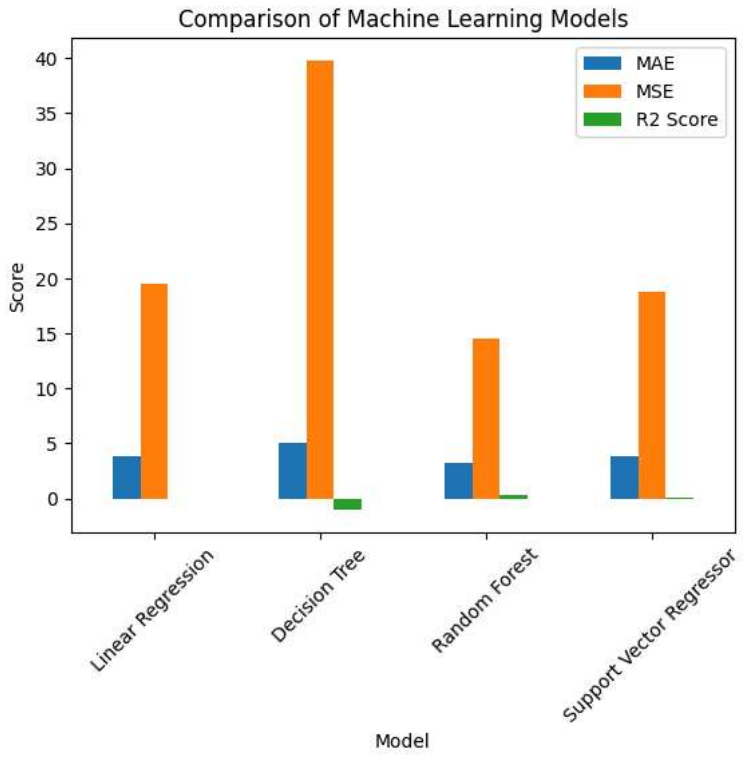



```
plt.figure()

plt.bar(comparison_df["Model"], comparison_df["MAE"])

plt.xlabel("Models")
plt.ylabel("Mean Absolute Error")
plt.title("Model Comparison based on MAE")

plt.xticks(rotation=45)

plt.show()
```




```
plt.figure()

plt.bar(comparison_df["Model"], comparison_df["MSE"])

plt.xlabel("Models")
plt.ylabel("Mean Squared Error")
plt.title("Model Comparison based on MSE")

plt.xticks(rotation=45)

plt.show()
```




```
comparison_df.set_index("Model").plot(kind="bar")
```

```
plt.title("Comparison of Machine Learning Models")

plt.ylabel("Score")

plt.xticks(rotation=45)

plt.show()
```